

GreenEvo

Plan nauki stacku technicznego

Uzasadnienie i przygotowanie do dyskusji o architekturze backendu Django

Materiał przygotowawczy do spotkania roboczego

Jak korzystać z tego dokumentu

Każda biblioteka jest opisana w trzech warstwach: (1) co to jest, (2) gdzie dokładnie używamy tego w GreenEvo -- zawsze z odniesieniem do konkretnej decyzji architektonicznej, nie w oderwaniu od kontekstu, (3) co konkretnie przeciwstawić, żeby umieć to pewnie wytłumaczyć i uzasadnić wybór.

Biblioteki są podzielone na trzy priorytety, żeby skupić czas nauki tam, gdzie się najbardziej opłaca:

TIER 1 -- opanuj płynnie

Opanuj płynnie -- rdzeń stacku, na tym stoi cały system. Pytania na spotkaniu najprawdopodobniej będą dotyczyć właśnie tych wyborów.

TIER 2 -- zrozum i przeciwstawić

Zrozum koncepcje i zrób prosty przykład -- ważne biblioteki, ale bardziej samoobsługowe. Wystarczy umieć wytłumaczyć decyzje i pokazać, że działają.

TIER 3 -- przeczytaj raz

Przeczytaj raz -- to w większości konfiguracja, nie wymaga głębokiej wiedzy koncepcyjnej.

Na końcu dokumentu: gotowe, krótkie odpowiedzi na najbardziej prawdopodobne pytania kontrujące ("dlaczego nie X zamiast Y").

Django -- fundament

TIER 1 -- opanuj płynnie

Django to dojrzały framework webowy w Pythonie z gotowym ORM, systemem migracji, panelem admina i mechanizmami bezpieczeństwa. W GreenEvo skupiamy się na trzech konkretnych elementach, nie na ogólnym odświeżaniu całego frameworka.

1. Custom User model -- zrob to jako pierwszy krok w projekcie

Potrzebujemy logowania po adresie e-mail, nie po nazwie użytkownika, oraz pol takich jak rola (Applicant/Administrator/Expert). To wymaga `AbstractBaseUser` + `PermissionsMixin` + własny manager, z `USERNAME_FIELD = "email"`. To trzeba zrobić PRZED pierwszą migracją -- dodanie tego później wymaga skomplikowanej migracji danych albo resetu bazy.

2. Management commands

`load_evaluation_model` (ładowanie modelu oceny z fixture'ow) będzie właśnie takim komendem: `BaseCommand`, `add_arguments()`, `handle()`.

3. Signals

`post_transition` z `django-fsm-2` (patrz Tier 1, pozycja 4) to nasz główny punkt integracji: audyt + powiadomienia w jednym miejscu, zamiast rozproszonej logiki po widokach.

PostgreSQL -- konkretnie JSONField

TIER 1 -- opanuj płynnie

SQL już znasz -- warto skupić się konkretnie na tym, jak Django używa Postgresa, nie na ogólnej nauce bazy danych.

Gdzie w GreenEvo

`Application.submitted_snapshot`, `AIAnalysisResult.raw_output`, `ApplicationSummary.content` -- wszystko to `django.db.models.JSONField`, mapowane na typ `jsonb` w Postgresie.

```
class AIAnalysisResult(models.Model):
    raw_output = models.JSONField()

# zapis i odczyt to zwykły dict/list w Pythonie:
result.raw_output = {"suggested_level": 3, "is_consistent": True}
result.save()
```

Na co uważać

`jsonb` jest indeksowalny (GIN index), ale przy skali kilku wniosków miesięcznie to zupełnie nieistotne teraz -- nie trzeba tego ogarniać na starcie projektu.

Django Ninja Extra (1/3)

TIER 1 -- opanuj płynnie

Caly nasz backend API -- kazdy endpoint z ARCHITEKTURA_DECYZJE.md (rozdz. 7) to jakis api_controller + metoda z @route. Schematy danych to zwykle modele pydantic (Schema), nie osobny system serializerow jak w DRF. Zmiana decyzji z DRF na Ninja Extra -- patrz rozdz. 2 ARCHITEKTURA_DECYZJE.md.

Nested schematy -- inaczej niz w DRF, ale nie trudniej

PATCH /applications/{id}/ musi zapisywac zagniezdzone applicant_info, eco_innovation_info i liste criterion_answers w JEDNYM requestcie. WAZNA ROZNICA wzgledem DRF: Schema (pydantic) to tylko kontrakt danych -- NIE MA automatycznego zapisu do bazy. Logike zapisu zawsze piszesz sam w ciele metody kontrolera.

```
class ApplicationInSchema(Schema):
    applicant_info: ApplicantInfoSchema

@api_controller('/applications')
class ApplicationController:
    @route.patch('/{app_id}')
    def update(self, app_id: int, data: ApplicationInSchema):
        app = get_object_or_404(Application, id=app_id)
```

Django Ninja Extra (2/3)

```
for attr, val in data.applicant_info.dict().items():
    setattr(app.applicant_info, attr, val)
app.applicant_info.save()
return app
```

Zrob mala probke z zagniezdzoneym schematem PRZED napisaniem wlasciwego kodu -- inny mechanizm niz DRF (brak auto-zapisu), warto to poczuc na malym przykladzie.

Custom routes na kontrolerze -- przejścia FSM

Dokladnie tak zrobimy wszystkie przejścia statusu (submit, assign-expert, approve-evaluation...):

```
@api_controller('/applications')
class ApplicationController:
    @route.post('/{app_id}/submit')
    def submit(self, app_id: int):
        application = get_object_or_404(Application, id=app_id)
        try:
            application.submit()
```

Django Ninja Extra (3/3)

```
except TransitionNotAllowed:  
    raise BusinessRuleError('INVALID_STATUS_TRANSITION', ...)  
return application
```

Permission classes -- object-level

Ninja Extra ma własny system permission classes, ale dla per-obiektowych uprawnień (IsOwnerApplicant / IsAssignedExpert) często prościej jest zrobić zwykły 'if not (...): raise PermissionDenied' wprost w metodzie kontrolera -- mniej ceremonii niż w DRF, mniej ubita ścieżka w dokumentacji.

Custom exception_handler

Rejestracja przez @api.exception_handler(BusinessRuleError) na poziomie NinjaExtraAPI -- to jak zrobimy spójny format błędów z kodami (RECRUITMENT_CLOSED, APPLICATION_LIMIT_REACHED, INVALID_STATUS_TRANSITION).

django-fsm-2 -- serce workflow wniosku (1/2)

TIER 1 -- opanuj płynnie

Najmniej standardowa biblioteka z całego stacku -- warto ją ogarniać szczególnie dobrze, bo zespół / Ministerstwo może o nią zapytać, a to ona odpowiada za cały workflow z 12 statusami.

```
from django_fsm import FSMField, transition

class Order(models.Model):
    status = FSMField(default='new', protected=True)

    @transition(field=status, source='new', target='paid')
    def pay(self): pass

    @transition(field=status, source='paid', target='shipped',
                conditions=[lambda self: self.has_stock()])
    def ship(self): pass
```

Co przeczwiczyć (15-20 minut, mała aplikacja) -- część 1

- Złap wyjątek `django_fsm.TransitionNotAllowed` przy przejściu z niewłaściwego stanu -- sprawdź jego atrybuty (`object`, `method_name`). Dokładnie to użyjemy w naszym exception handlerze do zbudowania czytelnego komunikatu.

django-fsm-2 -- serce workflow wniosku (2/2)

Co przecwiczyć -- czesc 2

- Napisz receiver na sygnał `django_fsm.signals.post_transition` -- to będzie miejsce zapisu `StatusTransition` i wysyłki powiadomien.
- `conditions=[...]` -- warunek biznesowy wpięty bezpośrednio w definicję przejścia (np. "czy nabor aktywny" dla `submit()`), zamiast ręcznego sprawdzania w widoku.
- `source` może być lista statusów: `source=[Status.DRAFT, Status.TO_COMPLETE]` -- dokładnie nasz przypadek dla `submit()`.

Podsumowanie decyzji

Zamiast ręcznego słownika przejść: `django-fsm-2` sam pilnuje niedozwolonych przejść, przejścia są czytelnymi metodami na modelu (samodokumentujące się), a sygnał `post_transition` daje jedno miejsce do audytu i powiadomien zamiast rozproszonej logiki po widokach.

django-ninja-jwt (1/2)

TIER 1 -- opanuj płynnie

JWT auth dla Django Ninja: access token (krotki czas zycia) + refresh token (dluzszy). Ten sam koncept co djangorestframework-simplejwt, dobrany pod zmieniony framework API (rozd. 2 ARCHITEKTURA_DECYZJE.md). Cal autoryzacja naszego API dziala na tym mechanizmie.

Uwaga -- to wymaga customizacji niezaleznie od wybranej biblioteki

Domyslne flow zwraca {access, refresh} w body JSON. U nas refresh token ma isc do httpOnly cookie (bezpieczniejsze niz localStorage po stronie React) -- trzeba wlasnej obslugi w kontrolerze logowania. To NIE jest wbudowane w zadna z tych bibliotek -- ten sam wzorzec trzeba by napisac recznie zarowno z simplejwt (DRF), jak i z ninja-jwt.

```
@api_controller('/auth')
class AuthController:
    @route.post('/token')
    def obtain_token(self, data: LoginSchema, response: HttpResponse):
        access, refresh = get_tokens_for_user(...)
        response.set_cookie('refresh_token', refresh,
                           httponly=True, secure=True, samesite='Lax')
        return {'access': access}
```

django-ninja-jwt (2/2)

Do przecwiczenia

- Config czasu zycia tokenow (odpowiednik SIMPLE_JWT w settings) -- sprawdz dokladne nazwy ustawien w dokumentacji ninja-jwt.
- Mechanizm blacklisty tokenow (jesli dostepny) -- potrzebny, zeby 'wylogowanie' realnie uniewazizalo refresh token (bez tego JWT jest bezstanowy).

Pytanie, ktore moga zadac: 'jak dezaktywujecie konto w trakcie aktywnej sesji?'

Odpowiedz: is_active=False blokuje nowe logowania, ale juz wydany access token jest wazny do wygasniecia -- stad krotki czas zycia access tokena (np. 15 min) jest swiadoma decyzja bezpieczenstwa, nie przeoczenie. To nie zalezy od wyboru biblioteki JWT -- sama natura tokenow.

TIER 2 -- zrozum i przećwicz

Automatyczne wersjonowanie zmian modelu -- dodajesz `history = HistoricalRecords()`, biblioteka sama tworzy tabele `Historical<Model>` i zapisuje snapshot przy każdym `save()`. Używamy na `Application`, `CriterionAnswer`, `ExpertReview`.

Na co uważać -- pytanie, które mogą zadać

'Po co dwa mechanizmy audytu?' Odpowiedź: `django-simple-history` śledzi zmiany POL (kto zmienił co i kiedy), `StatusTransition` to nasz własny log biznesowych przejść WORKFLOW. Działają równolegle, nie zastępują się -- każdy odpowiada na inne pytanie.

Do przećwiczenia

- Dodaj `HistoricalRecords()` do jednego modelu w toy-projekcie, zrób kilka zmian, sprawdź `instance.history.all()` i `instance.history.as_of(jakas_data)`.

TIER 2 -- zrozum i przećwicz

Kolejka zadań asynchronicznych działająca na tej samej bazie Postgres, bez Redis/RabbitMQ. Używamy do: maili, wywołan AI, generowania Excela, i łańcucha zadań po zatwierdzeniu raportu z wizytacji.

```
from django_q.tasks import async_task

async_task('myapp.tasks.run_ai_analysis', application_id,
           hook='myapp.tasks.on_analysis_done')

# hook wywoływany po zakończeniu -- tak łańcuchujemy zadania:
def on_analysis_done(task):
    async_task('myapp.tasks.build_summary', task.result['application_id'])
```

Do przećwiczenia: uruchom lokalnie 'python manage.py qcluster' (worker) i sprawdź, że zadanie faktycznie wykonuje się w tle, nie blokując requestu. Django-Q2 zapisuje wyniki w `django_q.models.Success/Failure` -- przyda się też do debugowania.

TIER 2 -- zrozum i przecwicz

Walidacja danych przez modele z typami -- ogólna biblioteka Pythona. U nas podwójna rola: walidacja JSON-a zwracanego przez model AI (ai_assist) ORAZ fundament Schema w całym Django Ninja Extra (rozdz. 2 ARCHITEKTURA_DECYZJE.md) -- ucz się tego tutaj, uczysz się jednocześnie podstawy całego API.

```
from pydantic import BaseModel, Field, ValidationError
class CriterionEvaluationResult(BaseModel):
    suggested_level: int | None = Field(ge=1, le=5)
    is_consistent: bool
    inconsistency_explanation: str
try:
    result = CriterionEvaluationResult.model_validate_json(llm_raw_output)
except ValidationError as e:
    errors = e.errors() # lista konkretnych błędów pol -- do 'jednej próby korekty'
```

Do przecwiczenia

- Różnica: `model_validate()` (z dicta) vs `model_validate_json()` (z surowego stringa JSON -- dokładnie to dostajemy z odpowiedzi LLM-a).
- Struktura `ValidationError.errors()` -- pomaga zbudować sensowny komunikat korekty wysyłany z powrotem do modelu AI przy nieudanej pierwszej próbie.

Konfiguracja -- przeczytaj raz (1/2)

TIER 3 -- przeczytaj raz

django-storages

Ustawiasz STORAGES w settings (Django 4.2+, dict z kluczami default/staticfiles), wskazujesz backend (FileSystemStorage lokalnie, S3Storage w chmurze). FileField/ImageField uzywaja tego transparentnie -- kod modelu sie nie zmienia miedzy srodowiskami.

openpyxl

```
from openpyxl import Workbook
wb = Workbook()
ws = wb.active
ws.append(['Nazwa wniosku', 'Punktacja A', 'Punktacja B', 'Punktacja C'])
for app in applications:
    ws.append([app.name, app.score_a, app.score_b, app.score_c])
wb.save('export.xlsx') # albo do BytesIO na odpowiedz HTTP
```

Tyle wlasciwie wystarczy do MVP -- nie trzeba znac zaawansowanego formatowania.

Konfiguracja -- przeczytaj raz (2/2)

django-environ

```
import environ
env = environ.Env()
DEBUG = env.bool('DEBUG', default=False)
DATABASES = {'default': env.db('DATABASE_URL')}
```

Jedno zadanie: nie trzymać sekretów w kodzie.

pytest-django + factory_boy

```
@pytest.mark.django_db
def test_submit_application(applicant_user):
    app = ApplicationFactory(applicant=applicant_user, status='draft')
    app.submit()
    assert app.status == 'submitted'
```

factory_boy generuje dane testowe automatycznie (factory.Faker) -- oszczędza pisanie boilerplate'u w każdym teście.

~~drf-spectacular~~ -- niepotrzebne

Po zmianie na Django Ninja Extra dokumentacja OpenAPI (/api/docs/) generuje się automatycznie z type hintów kontrolerów i schematów pydantic -- nie trzeba osobnej biblioteki ani ręcznych adnotacji na custom endpointach. Jeden punkt mniej do nauczenia się.

Jesli beda pytac "dlaczego nie X" (1/2)

■ Dlaczego nie Celery zamiast Django-Q2?

Przy skali kilku wnioskow miesiecznie osobny broker (Redis/RabbitMQ) to zbedna infrastruktura do utrzymania -- Django-Q2 dziala na tej samej bazie Postgres. Migracja do Celery jest mozliwa pozniej, gdyby skala istotnie wzrosla.

■ Dlaczego nie reczny slownik przejsc zamiast django-fsm-2?

Sam pilnuje niedozwolonych przejsc i jest czytelniejszy -- metody na modelu zamiast rozproszonej logiki w widokach, latwiejszy do testowania i utrzymania.

■ Dlaczego nie django-viewflow zamiast django-fsm-2?

Viewflow to pelny silnik BPM zaprojektowany pod server-rendered UI (widoki zadan, Django admin) -- nie pasuje do architektury z osobnym frontendem React. Nasz workflow (12 statusow, liniowy z jedna petla i jednym rozgalezieniem) nie potrzebuje pelnego BPM engine z rownoleglymi galeziami/join -- fsm-2 jest proporcjonalny do rzeczywistej zlozonosci.

Jesli beda pytac "dlaczego nie X" (2/2)

■ Dlaczego Django Ninja Extra a nie Django REST Framework?

Spojność: i tak potrzebujemy pydantic do walidacji odpowiedzi AI, więc Ninja daje jeden mechanizm walidacji w całym projekcie zamiast dwóch. Do tego automatyczny OpenAPI bez ręcznych adnotacji -- istotne, bo znaczna część API to custom-action endpointy (przejścia FSM). Świadomie akceptujemy mniejszy ekosystem/społeczność niż DRF.

■ Dlaczego JWT a nie sesje?

Frontend (React) i backend to rozdzielone aplikacje -- JWT jest bezstanowym, standardowym rozwiązaniem dla takiej architektury.

■ Dlaczego PostgreSQL a nie coś innego?

Decyzja przyjęta wprost na starcie projektu. Dobrze pasuje do potrzeb audytowalności (transakcyjność, integralność referencyjna) i obsługuje JSONField dla wyników AI.

■ Dlaczego model oceny jest 'danymi', a nie zaszyty w kodzie?

Wagi kryteriów zostaną zaktualizowane po badaniu eksperckim metodą ABCD Suzuki -- architektura musi to znieść bez migracji i redeployu kodu.